MCP Gateway Governance on Azure AKS: Fine-Grained Per-Role, Per-Tool Access Control — A Curated Research Report

TL;DR

- **For your exact scenario, IBM ContextForge (`mcp-context-forge`) is the best-fit open-source MCP gateway today** — it ships "virtual servers" (curated tool subsets composed from multiple backends), private/team/public visibility, RBAC that filters `tools/list` per authenticated team, native Microsoft Entra ID SSO with group→team/role mapping, and an official Helm chart tested on AKS. **Solo.io's agentgateway** (Linux Foundation, Rust, Kubernetes-native, Cedar/CEL tool-level RBAC) is the strongest runner-up if you want a mesh-native data plane; **Azure API Management** is the best Azure-native option but only enforces tool filtering via hand-written policy, not a role/virtual-server model.
- **The governance you want ("Platform Engineer sees most Spark+Airflow tools but zero OpenMetadata; Data Analyst sees OpenMetadata + a few Spark tools + zero Airflow") requires gateway-side filtering at BOTH `tools/list` (discovery) and `tools/call` (invocation).** Filtering discovery alone is not enough: research (arXiv:2605.18414) found that with no guidance, models select forbidden tools at rates between 48% and 68.5%, and up to 37.0% even when given an explicit per-tool allowlist — so the invocation path must be independently authorized. This is a solved pattern in ContextForge (virtual servers + team-scoped tokens), agentgateway (CEL/Cedar RBAC on `mcp.tool.name`), and APIM (allowlist policy that synthesizes the tool catalog and hard-blocks disallowed calls).
- **Anchor auth on the MCP authorization spec (OAuth 2.1 + PKCE, Protected Resource Metadata RFC 9728, Resource Indicators RFC 8707) with Entra ID as the authorization server, and enforce the two named MCP security rules: no token passthrough and per-request audience validation to prevent the confused-deputy attack.** Map the Entra `roles`/`groups` claim to gateway roles, and use AKS Workload Identity for the gateway pod's access to Cosmos DB (data-plane RBAC), Application Insights, and ACR.

Key Findings

1. The MCP authorization spec has matured fast and is now enterprise-ready. As of the current in-force revision (2025-11-25), remote HTTP MCP servers are OAuth 2.1 *resource servers* only; a separate authorization server (Entra ID) issues tokens. Mandatory building blocks: OAuth 2.1 Authorization Code flow with PKCE (S256 only), Protected Resource Metadata (RFC 9728, `/.well-known/oauth-protected-resource`), Authorization Server Metadata (RFC 8414) or OIDC Discovery, and Resource Indicators (RFC 8707) so tokens are bound to a specific MCP server audience. The June 2025 revision introduced the resource-

server split; November 2025 added Client ID Metadata Documents (CIMD, downgrading Dynamic Client Registration workos from SHOULD to MAY), step-up authorization via `WWW-Authenticate`, stackoverflow and Cross-App Access (XAA) for enterprise-managed authorization. auth0

2. Tool-level RBAC/ABAC is a gateway concern, and it must gate both discovery and invocation. The standard MCP protocol has no built-in mechanism to filter which tools are visible per user — that is exactly the gap gateways fill. Best-practice enforcement: (a) filter the `tools/list` JSON-RPC response so the model never sees forbidden tools (keeps context lean and avoids prompting the model toward tools it can't use), and (b) independently authorize `tools/call` so a hallucinated or injected tool name can't bypass the discovery filter. arxiv RBAC (role→tool-set) covers your scenario; ABAC (attributes: user, resource tags, environment, time) via policy engines (OPA/Rego, Cedar, Cerbos) adds context-aware conditions.

3. "Virtual servers" / "scoped views" are the cleanest pattern for your data-team example. Instead of standing up parallel MCP servers per permission level, a gateway composes a virtual MCP server from a hand-picked subset of tools drawn across multiple backends, then grants a team/role access to only that virtual server. ContextForge and agentgateway both implement this; it is the recommended way to express "Platform Engineer VS = most Spark + Airflow tools; Data Analyst VS = OpenMetadata + a few Spark tools."

4. There is a Microsoft-published open-source `microsoft/mcp-gateway` that matches your exact AKS topology — but its governance is coarser than ContextForge's. It is a .NET reverse proxy/control plane for MCP servers on Kubernetes with session-aware routing, and its 1-click Azure Bicep deploy provisions precisely your stack: AKS, ACR, Cosmos DB, Application Gateway, Application Insights, VNet, and Managed Identity. Its authorization model is Entra ID bearer-token auth with app-role-based read/write access at the adapter/tool level (`mcp.admin`, `mcp.engineer`, plus creator ownership) — not per-role curated tool views. Use it for the plumbing/topology reference; use ContextForge or agentgateway for the fine-grained governance layer.

5. A critical caveat from the security community: tool-name allowlisting is necessary but not complete access control. When a single MCP tool (e.g. `call_aws`, `execute_sql`, `soqlQuery`) hides many underlying operations or accepts batch commands, nhimg a gateway that only sees "one JSON-RPC envelope in, one result out" cannot authorize the individual downstream operations. p0 For truly sensitive backends, pair gateway RBAC with least-privilege, short-lived, session-scoped credentials enforced by the backend's own IAM (via OAuth 2.0 Token Exchange, RFC 8693), rather than assuming tool-level filtering equals authorization.

Details

1. Foundational knowledge: the MCP authorization/authentication spec

Spec timeline and what changed (official + high-quality secondary sources):

- **2024-11-05** — initial MCP spec; defines client-server architecture, JSON-RPC 2.0, tools/resources/prompts, stdio + HTTP+SSE transports. (hidekazu-konishi)
- **2025-03-26** — adds the OAuth 2.1 authorization framework, replaces HTTP+SSE with Streamable HTTP (session via `Mcp-Session-Id`), (hidekazu-konishi) requires PKCE for all clients.
- **2025-06-18** ("The Great Separation") — MCP servers are formally OAuth 2.1 *resource servers* only; mandates Protected Resource Metadata (RFC 9728) and Resource Indicators (RFC 8707); adds a dedicated security-best-practices page.
- **2025-11-25** (currently in force) — bans plain PKCE (S256 only), adds OIDC Discovery 1.0 as an alternative (mcpcn) to RFC 8414, introduces Client ID Metadata Documents (CIMD) (workos) and downgrades Dynamic Client Registration from SHOULD to MAY, adds step-up (incremental scope) authorization via `WWW-Authenticate`, (stackoverflow) Cross-App Access (XAA), URL-mode elicitation, and experimental async tasks. (daily)
- Note: several third-party blogs reference later dates (a "2026-07-28" revision) as forthcoming/tightening; treat anything beyond 2025-11-25 as draft/forward-looking, since 2025-11-25 is the latest ratified revision as of this report.

The RFC stack you must implement at the gateway: OAuth 2.1 (draft) Authorization Code + PKCE (RFC 7636), JWT (RFC 7519), Resource Indicators (RFC 8707), Authorization Server Metadata (RFC 8414), Protected Resource Metadata (RFC 9728, April 2025), Dynamic Client Registration (RFC 7591, now MAY), OAuth 2.0 Security BCP (RFC 9700), and Token Exchange (RFC 8693) for downstream calls.

How this maps to Entra ID: Entra ID acts as the authorization server; the gateway (or APIM in front of it) validates JWTs, checks the `aud` claim equals the MCP server identifier on every request, and reads the `roles` (app roles) or `groups` claim to drive tool-level RBAC. Microsoft's own guidance recommends app roles over the groups claim because app roles avoid the "group overage" problem, are scoped to the application, and travel with the token.

2. The specific gateways — capability comparison

IBM ContextForge (`IBM/mcp-context-forge`) — best fit for your requirements (open-source, self-hosted, Apache-2.0).

- **Virtual servers (shipped, core feature):** "A Virtual Server is a MCP Server composed from Tools/Resources/Prompts from multiple servers. Add one or more MCP Servers under 'Gateways', then select which Tools/Prompts/Resources to use." (github) (github)

You compose a curated subset via the `associated_tools` list when calling `POST /servers`. This directly implements your Platform-Engineer-VS vs Data-Analyst-VS split.

- **RBAC (shipped, GA in the 1.0.x line):** Two-layer model — Layer 1 token/team scoping filters what a user *can see* (applied to `tools/list` **before** RBAC), Layer 2 `@require_permission` decorators control what a user *can do*. github Four built-in roles: Platform Admin (global, `*`), Team Admin, Developer (`tools.read`, `tools.execute`, …), Viewer (read-only). github Resources (tools, servers, prompts) have private/team/public visibility, each owned by a `team_id` + `owner_email`. `tools/list` is filtered per authenticated team; a missing `teams` claim defaults to public-only (secure default). github Token-scoped list filtering was enforced in v1.0.0-RC1 (#2663) and hardened through v1.0.x.
- **Per-virtual-server scoped tokens (shipped):** API tokens can be restricted to specific virtual servers ("server scoping") and/or specific teams ("team scoping"), with IP restrictions, expiry, usage tracking, and revocation; github scope enforcement hardened in v1.0.7 (#4737).
- **Entra ID SSO (shipped):** Supported SSO providers include GitHub, Google, **Microsoft Entra ID**, IBM Security Verify, Okta, Keycloak, generic OIDC. Config via `SSO_ENTRA_TENANT_ID` / `SSO_ENTRA_CLIENT_ID` / `SSO_ENTRA_CLIENT_SECRET` github / `SSO_ENTRA_ADMIN_GROUPS`. Entra group→role and group→team mapping shipped via feature #2054; team mapping configured through Admin UI or API (`team_mapping` JSON). Caveat: a known Entra group-count fetch limit (#2201), and Entra emits group object-ID GUIDs by default.
- **AKS/Helm (shipped):** Official `charts/mcp-stack` Helm chart explicitly lists AKS as a target; deploys gateway (n replicas) + PostgreSQL + Redis github with an automatic Alembic migration job; OCI chart at `oci://ghcr.io/ibm/mcp-context-forge` usable with Argo CD/Flux; github dedicated Azure/AKS deployment guide exists.
- **Version status:** multi-tenancy/teams/visibility introduced in v0.7.0; RBAC epic #283; past 1.0 GA. As of this report the latest release is **v1.0.10 (Sep 7, 2026)** per the PyPI `mcp-contextforge-gateway` release history (v1.0.9 Sep 1, 2026; v1.0.8 Aug 18, 2026). Roadmap (NOT shipped): ABAC for virtual servers, role auto-assignment, OAuth 2.0 Token Exchange (#3385) github and Client-Credentials M2M (#3386) for upstream brokering. github Observability via OpenTelemetry (OTLP) → Jaeger/Tempo/DataDog/New Relic and others. github

Solo.io agentgateway (`agentgateway.dev`, Linux Foundation, Rust) — strongest Kubernetes-native runner-up.

- Purpose-built AI-native data plane (LLM + MCP + A2A). Federates multiple MCP servers behind one endpoint; can virtualize/compose MCP tool servers ("Virtual MCP").

- **Tool-level RBAC:** JWT validation against a JWKS provider, then CEL-based policy on tool access. (shanedeconinck) Per the Solo.io "Control access to tools" docs, "By default, all tool access is allowed"; rules use an `EnterpriseAgentgatewayPolicy`/`AgentgatewayPolicy`, e.g. `jwt.sub == "alice" && mcp.tool.name == "add_issue_comment"` means "JWT tokens with the `sub=alice` claim can access the `add_issue_comment` tool. Access to all other tools is denied." Authorization "runs at two points: 1. During discovery, it removes capabilities the client cannot use from MCP list responses. 2. Before execution, it authorizes each tool call … again"; rules are OR-ed and "if a tool matches none of them, it is denied." Also supports Cedar (Amazon's RBAC/ABAC/ReBAC policy language) for fine-grained principal/action/resource rules. (shanedeconinck)
- Configured via Kubernetes CRDs (`AgentgatewayPolicy` / `EnterpriseAgentgatewayPolicy`); integrates with any OIDC IdP (Entra ID via standard OIDC), mesh mTLS, OpenTelemetry. Featured in Microsoft's own three-layer LLM-routing-on-AKS reference architecture (2026). (infoq) Caveat: early in its Linux Foundation lifecycle — validate RBAC maturity and protocol coverage for your environment.

Azure API Management — best Azure-native option (managed, deep Entra integration).

- Exposes REST APIs as MCP servers (operations become tools) or fronts an existing MCP server (must be MCP 2025-06-18+). Governance via APIM policies: JWT validation (Entra ID or other IdP), rate limiting/quotas, IP filtering, caching. (microsoft)
- **Key limitation for your use case:** "Currently, policies apply to all API operations exposed as tools in the MCP server" (microsoft) (microsoft) — there is no built-in per-role/per-tool model. You achieve tool filtering by hand-written policy. The Microsoft blog "Controlling Tool Access with APIM MCP Gateway" documents two approaches: **Option A (static allowlist)** — APIM intercepts `tools/list` and returns a hardcoded catalog, and blocks `tools/call` for anything off-list; (microsoft) **Option B (dynamic deny-list)** — reads/rewrites the response body (with the caveat that response-body access can interfere with streaming). (microsoft)
- **Per-role tool access** is achievable by declaring Entra app roles on the MCP API app registration and enforcing the `roles` claim in a *per-server* policy — Microsoft's "From Simple to Advanced" blog shows publishing multiple MCP servers (e.g., read-only vs read-write) from one backend API and gating privileged servers on the app role. This is the APIM-native equivalent of virtual servers, but you maintain the policies by hand.
- **`tools/list` gotcha:** by default the `tools/list` endpoint does not enforce subscription-key validation, so tools can be enumerated without auth (itsrene) unless you add a policy. Also: don't read `context.Response.Body` in MCP policies generally (breaks streaming). (azure) APIM currently supports MCP tools (microsoft) (and, for external servers, resources) but not prompts. **Microsoft Foundry also now offers "Govern MCP tools by**

using an AI gateway (preview)", routing new Foundry MCP tools through an APIM-backed gateway. (microsoft)

Docker MCP Gateway / MCP Toolkit — good for local/dev and container isolation, weaker on per-role RBAC.

- Runs each MCP server in an isolated container; unifies them behind one gateway; supports OAuth token management, signature checks, secret isolation, and interceptors. (docker) **Tool allowlisting is per-profile**, not per-user-role: (`docker mcp profile tools --enable github.create_issue`), (`--disable-all github`), etc. (github) You can run multiple gateways/profiles for different tool sets, but there is no built-in identity→role→tool mapping across users; community practice is a shared gateway with catalog-based filtering.

Lasso Security MCP Gateway (`lasso-security/mcp-gateway`, open source, Python) — security/guardrails-first, not RBAC-first.

- Plugin-based gateway orchestrating other MCPs; strengths are guardrails (PII/token masking via Presidio, Lasso threat API), a security scanner that scores MCP-server reputation and detects tool-poisoning/hidden instructions, and auto-blocking risky servers. It partners with Portkey (whose MCP Gateway provides the access-control/policy control plane) for enterprise deployments. Choose it as a *guardrail layer* alongside a governance gateway, not as your primary per-role tool-RBAC engine.

Other notable options:

- **Portkey MCP Gateway** — access control that scopes internal systems tightly, blocks risky tool calls, and changes permissions without redeploying; a single control plane over LLM + MCP traffic.
- **Kuadrant** (`mcp-gateway`) (CNCF Sandbox, Envoy-based) — aggregates MCP servers; (codecentric) Red Hat's guide shows identity-based (`tools/list`) filtering via a signed (`x-authorized-tools`) "wristband" JWT (Authorino extracts permissions), plus OAuth2 Token Exchange and Vault integration.
- **Envoy AI Gateway** — MCP gateway with stateless session management, token-aware rate limiting. (codecentric)
- **Commercial/managed:** TrueFoundry (dedicated RBAC + virtual MCP servers), MintMCP (SCIM-driven RBAC, agent bundles), Bifrost/getmaxim (MCP tool groups via virtual keys), Natoma, Datawiza Agent Gateway (Entra-ID-front-of-MCP with tool/action policy), Amazon Bedrock AgentCore Gateway. AgentCore uses Cedar policies keyed on Principal (agent/user identity), Action (the MCP tool name), Resource (tool target), and Conditions (runtime context, e.g. time of day, read-only flag) — for example, (`forbid(principal, action == Action::"tools/call", resource ==`

`Tool::"delete_patient_record") unless { principal.role == "admin" }` — with OPA/Rego added inside the request path for rules Cedar can't natively express (time windows, payload inspection, change-ticket requirements).

3. Newer updates & best practices (2025–2026)

MCP security — the named risks you must design against (Anthropic spec + security community):

- **Confused deputy:** a privileged MCP server/gateway is tricked into misusing its authority for an attacker. Mitigate with per-client consent registries, explicit user consent for dynamically registered clients, and strict redirect-URI validation.
- **Token passthrough (explicitly forbidden by the spec):** never forward the client's inbound token to downstream services. The gateway must validate tokens directly and use OAuth 2.0 Token Exchange (RFC 8693) to obtain a *new*, audience-scoped, short-lived credential for each downstream MCP server/tool.
- **Audience validation on every request:** reject any token whose `aud` doesn't match the MCP server identifier, even if signed and unexpired.
- **Session security:** cryptographically random, non-deterministic session IDs bound to user identity (e.g. `user_id:session_id`); never use sessions for authentication.
- **Sandboxing/supply chain:** containerize MCP servers with default-deny egress, pin/sign images, scan tool descriptions for hidden instructions (tool poisoning/rug-pull). Microsoft's `mcp-for-beginners/02-Security` recommends Entra ID, Prompt Shields/Azure Content Safety, `github` and the OWASP-aligned controls. There is also a `microsoft/agent-governance-toolkit` "MCP Security Gateway" spec (draft) covering interception, response scanning, and audit.

Emerging patterns: "scoped virtual MCP servers per team," "identity-aware MCP proxy," gateway-enforced tool allowlisting at both discovery and invocation, and just-in-time/session-scoped credentials. Academic work — Rohith Uppala, "Prompts Don't Protect: Architectural Enforcement via MCP Proxy for LLM Tool Access Control" (arXiv:2605.18414, v3 27 Aug 2026, accepted to EMNLP 2026 Industry Track) — demonstrates a governed MCP proxy doing ABAC at both `tools/list` and `tools/call` with 1.72 ms median end-to-end overhead across 1,000 requests (JWT verify 0.04 ms + attribute auth 0.002 ms + MongoDB query 1.59 ms + ABAC defense filter 0.05 ms), "negligible relative to LLM inference latency (typically 500ms–30s)." The same paper found that with no guidance, models "select forbidden tools at rates between 48% and 68.5%," and "up to 37.0% even when given an explicit per-tool allowlist," whereas the architecturally enforced proxy yields "0% Unauthorized Invocation Rate by design" — the empirical case for enforcing at the invocation path, not via prompts. KubeCon EU 2026, RSAC 2026, and the AAIF MCP Dev Summit all flagged MCP governance ("shadow MCP," OWASP MCP Top 10) as the current production gate.

4. Deployment / implementation on AKS

Reference topology (matches your diagram): App Gateway (WAF) → internal LB → gateway pod in VNet → backend MCP server pods; telemetry → Application Insights; session/metadata → Cosmos DB; images ← ACR. Microsoft's AKS baseline and advanced-microservices reference architectures cover the surrounding controls: Microsoft Entra Workload ID for pods, ingress/egress restrictions, Azure RBAC for Kubernetes, and managed-identity access to ACR/Key Vault.

Identity plumbing (Azure-native, credential-less):

- **AKS Workload Identity** (OIDC federation; preconfigured in AKS Automatic) (microsoft) lets the gateway pod authenticate to Entra-protected Azure services with no secrets in-cluster.
- **Cosmos DB:** use **data-plane RBAC** (Cosmos DB Data Reader/Contributor roles for the pod's managed identity) — note this is distinct from Azure control-plane RBAC; assigning only an Azure role yields 403s on document reads. Partition per-user/session data by the Entra (`oid`) claim (stable) — not (`sub`). Disable local key auth (`--disable-local-auth true`) once migrated. (cloudwebschool)
- **Application Insights / ACR:** kubelet identity authenticates to ACR; use OpenTelemetry from the gateway to App Insights.

Working code / IaC:

- (`microsoft/mcp-gateway`) — .NET gateway, K8s manifests under (`deployment/k8s/`), and **Bicep 1-click Azure deploy** provisioning AKS + ACR + Cosmos DB + App Gateway + App Insights + Managed Identity + VNet. Entra app-role setup documented in (`docs/entra-app-roles.md`).
- (`Azure-Samples/AI-Gateway`) — 30+ hands-on labs (Jupyter + **Bicep + APIM policies**) including "Model Context Protocol (MCP)," "MCP Client Authorization," and Realtime+MCP.
- (`microsoft/MultiAgent-Accelerator`) — MAF + A2A + MCP on AKS/ACA with deployment scripts and per-agent allowed-tool lists.
- ContextForge (`charts/mcp-stack`) Helm chart + Azure/AKS deployment guide.
- (`Azure/aks-mcp`) — official MCP server (image (`ghcr.io/azure/aks-mcp:latest`)) (medium) for operating AKS itself.

5. Learning resources

- **Official MCP spec & security:** (`modelcontextprotocol.io/specification/2025-11-25`) (authorization + security-best-practices pages); changelog at (`modelcontextprotocol.info/specification/2025-11-25/changelog`).

- **Microsoft official:** `learn.microsoft.com` — "Overview of MCP servers in Azure API Management," "Expose REST API as MCP server," "Connect and Govern Existing MCP server," "Govern MCP tools by using an AI gateway (preview)" (Foundry), AKS baseline/advanced architectures, Workload Identity. Tech Community blogs: "MCP in Azure: Using API Management for Authentication, Access, Logging & Governance," "MCP Server Authorization with Azure API Management: From Simple to Advanced," "Controlling Tool Access with APIM MCP Gateway." `github.com/microsoft/mcp-for-beginners` (Security module) and `microsoft/mcp` (catalog).
- **ContextForge:** docs at `ibm.github.io/mcp-context-forge` (multitenancy, RBAC, teams, virtual servers, Entra tutorial, Azure/Helm deploy); Niklas Heidloff's "Managing MCP Tools with Context Forge"; `heidloff` maintainer Mihai Criveti's Medium walkthrough.
- **agentgateway / Solo.io:** `agentgateway.dev/docs` "Tool access" (CEL RBAC), Solo.io blogs on Virtual MCP and zero-trust agentic config with kagent; KubeCon EU 2026 sessions (Kyverno + agentgateway MCP policy demos). `solo`
- **Conference talks (2026):** Microsoft Build 2026 MVP lightning talks (e.g., "Build Production MCP Servers on Azure Functions," LTG455; "Building your own MCP server," LTG466 `microsoft` — YouTube via aka.ms links); KubeCon EU 2026 recaps (codecentric, Kubermatic) on AI gateways and MCP governance.
- **Security/analysis:** P0 Security "From MCP Tool Filtering to Runtime Access Control," Aembit and Wiz MCP threat-modeling guides, arXiv 2511.20920 ("Securing the MCP: Risks, Controls, and Governance") and arXiv:2605.18414 ("Prompts Don't Protect").

Recommendations

Stage 0 — Decide the governance engine (this week). Adopt **IBM ContextForge** as your MCP gateway if you want the richest built-in per-role/per-tool model with Entra ID SSO and a supported AKS Helm chart. Choose **agentgateway** instead if you are committed to a Kubernetes/CRD-native, mesh-integrated data plane and are comfortable with a younger project and CEL/Cedar policy authoring. Use **APIM** if organizational policy mandates an Azure-managed, fully-supported service and you accept authoring tool-filtering policies by hand. Do **not** rely on `microsoft/mcp-gateway` for fine-grained per-role tool views — use it only as the AKS topology/IaC reference.

Stage 1 — Prove the data-team scenario in a PoC. With ContextForge: (1) register OpenMetadata, Spark, and Airflow MCP servers as "Gateways"; (2) create two virtual servers — `platform-eng` (`associated_tools` = most Spark + Airflow tools, zero OpenMetadata) and `data-analyst` (OpenMetadata + the few permitted Spark tools, zero Airflow); (3) set each virtual server to `visibility=team`, owned by the matching team; (4) enable Entra ID SSO and map the Entra groups `Platform-Engineers`→platform-eng team and `Data-Analysts`→data-analyst team; (5) verify that `tools/list` returns only permitted tools per user and that

`tools/call` on a disallowed tool is rejected. **Benchmark that would change this:** if `tools/list` filtering works but `tools/call` is not independently blocked, treat the product as insufficient.

Stage 2 — Harden auth. Configure Entra ID as the OAuth 2.1 authorization server; enforce PKCE (S256), audience validation (`aud` = MCP server ID) on every request, and Resource Indicators. **Forbid token passthrough**: have the gateway mint downstream credentials via Token Exchange (RFC 8693). Prefer Entra **app roles** over the groups claim to avoid group overage.

Stage 3 — Wire the Azure plane. Deploy on AKS with **Workload Identity**; give the gateway pod **Cosmos DB data-plane RBAC** (Data Contributor) and partition session/metadata by the Entra `oid` claim; disable Cosmos local-auth keys; send OpenTelemetry traces to Application Insights; pull images from ACR via kubelet identity; front everything with App Gateway (WAF) → internal LB, matching your topology.

Stage 4 — Add guardrails and defense-in-depth. Layer a guardrail/scanner (Lasso plugin, Azure Content Safety/Prompt Shields) for tool-poisoning and PII; containerize backend MCP servers with default-deny egress and signed/pinned images; enable audit logging of every `tools/call` (who, which tool, when). For high-sensitivity backends, add just-in-time, session-scoped downstream credentials rather than relying on tool-name allowlisting alone.

Thresholds that change the plan: if a single backend tool fans out into many privileged downstream operations (e.g., a generic `execute_sql`), `nhimg` do not treat gateway tool-filtering as authorization — push enforcement into that backend's native IAM. If you exceed the Entra group-fetch limit in ContextForge, switch to app-role claims. If you need ABAC (time-of-day, data-tag, environment conditions) beyond role→tool mapping, add OPA/Rego or Cedar/Cerbos as a policy decision point (agentgateway has Cedar natively; ContextForge lists ABAC-for-virtual-servers as roadmap).

Caveats

- **Fast-moving space:** MCP spec revisions, gateway features, and versions change monthly. The latest ratified spec revision is 2025-11-25; blog references to a "2026-07-28" revision describe draft/forthcoming behavior — verify against `modelcontextprotocol.io` before committing.
- **Vendor content bias:** many capability claims come from vendor blogs (Solo.io, Portkey, Lasso, TrueFoundry, MintMCP, getmaxim). Treat comparative superlatives (e.g., agentgateway's throughput/latency numbers) as vendor-reported, and validate in your environment.
- **ContextForge roadmap vs shipped:** its own FAQ "Roadmap" section is partially stale and still lists RBAC and per-virtual-server API keys as planned, while dated 1.0.x release

notes show them shipped. I prioritized the release notes and the multitenancy design doc; still, confirm the exact behavior on the version you deploy (latest v1.0.10, Sep 7 2026).

- **APIM's per-tool control is DIY:** APIM has no native role→virtual-server model; you build tool filtering and per-role gating with policy expressions, and must watch the `tools/list` no-auth default and the response-body/streaming caveat.
- **Tool-filtering ≠ full authorization:** the P0 Security critique is important — for opaque/batch/fan-out tools, gateway allowlisting alone cannot express the real authorization boundary; combine with backend IAM and short-lived scoped credentials. nhimg
- `microsoft/mcp-gateway` **governance is coarse** (app-role read/write at adapter/tool level, creator ownership) and its Agents/Sessions subsystem is explicitly preview/single-replica with in-pod tool execution — not for multi-tenant production without an added sandbox.